

BASES DE DATOS

BBDD no relacionales

Guillermo Granger Reguera

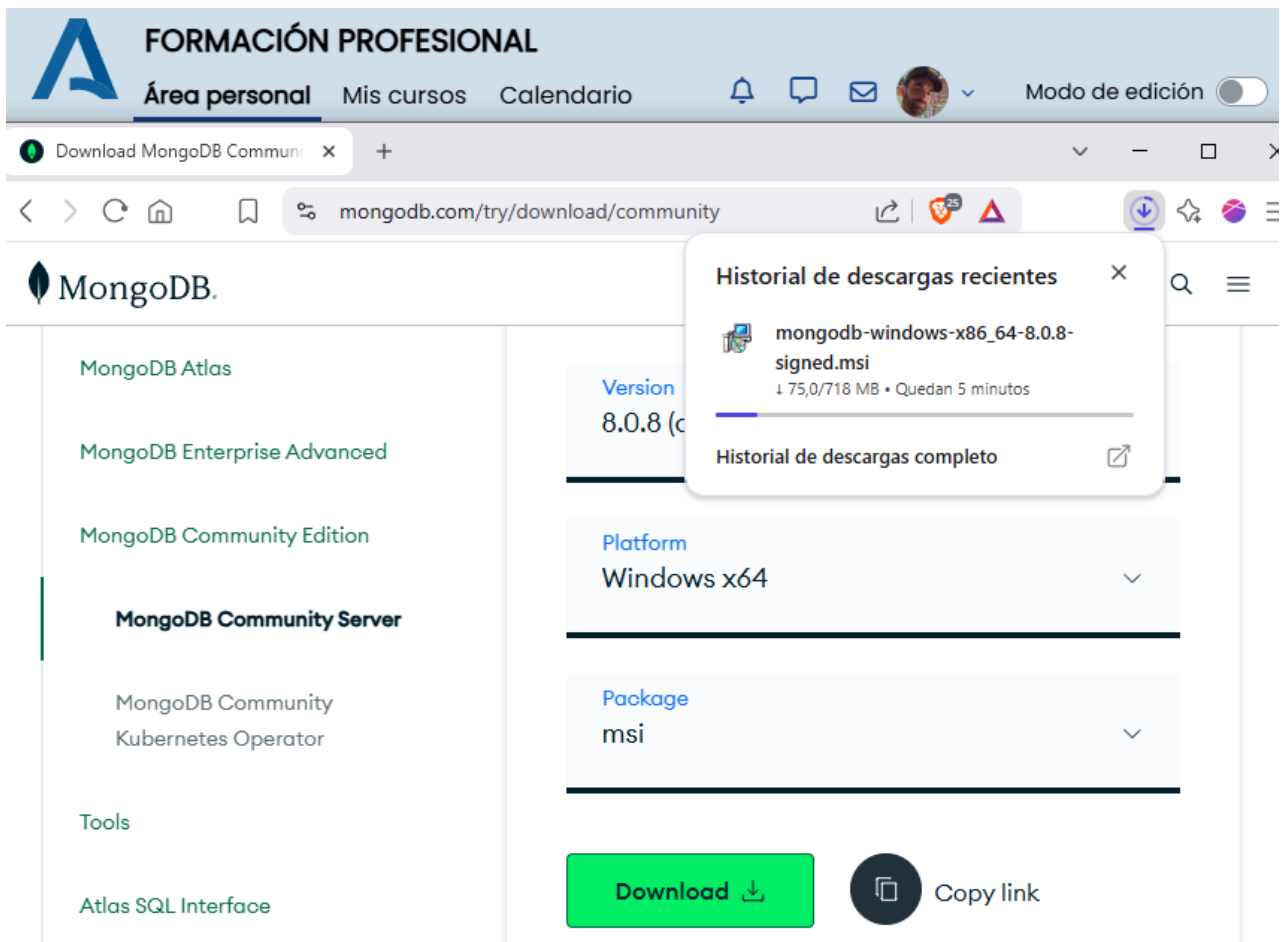
DAW/AGUA2 - Bases de datos - Grupo C

INDICE:

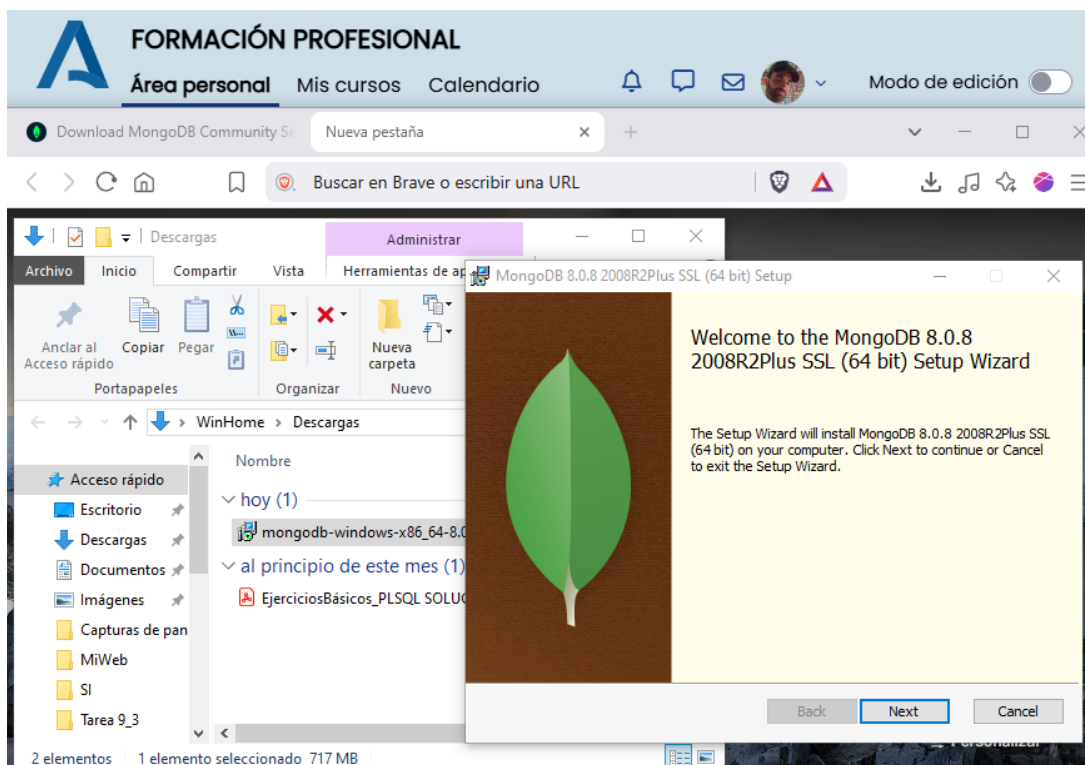
- Apartado Apag. 3
- Apartado B pag. 11

Apartado A: Guía de instalación y primeros pasos. Instalación de *MongoDB* (que incluye el servidor y *Compass*)

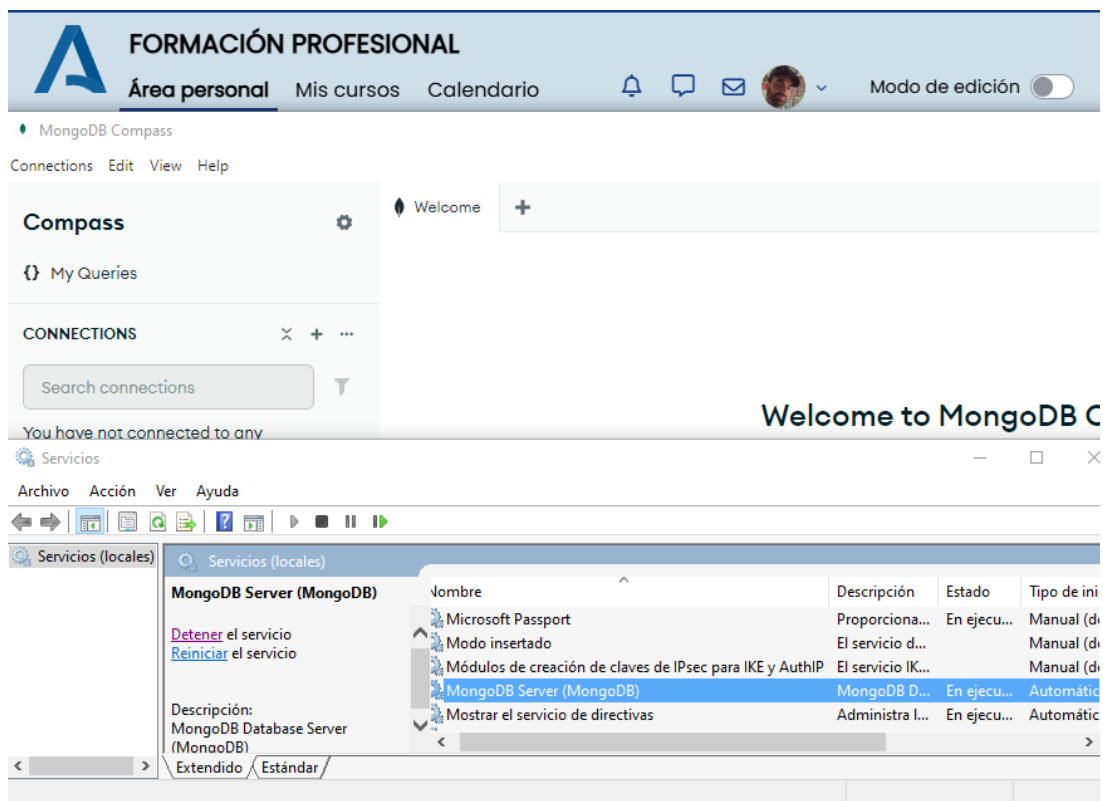
1. Se procede a la descarga del producto **MongoDB Community Server** para el sistema operativo Windows 10. Se ha descargado de la web oficial de MongoDB:



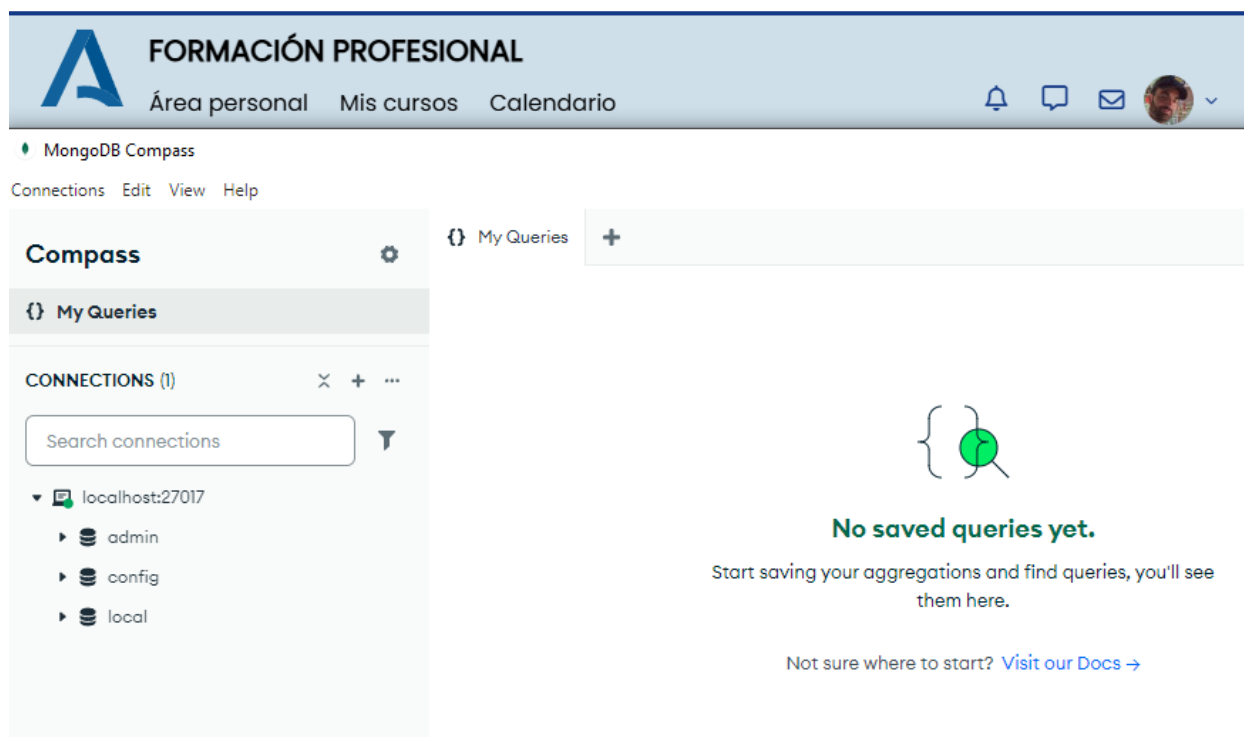
2. Se ejecuta el fichero descargado en mi carpeta Descargas y se realiza una instalación completa siguiendo las instrucciones del instalador de MongoDB:



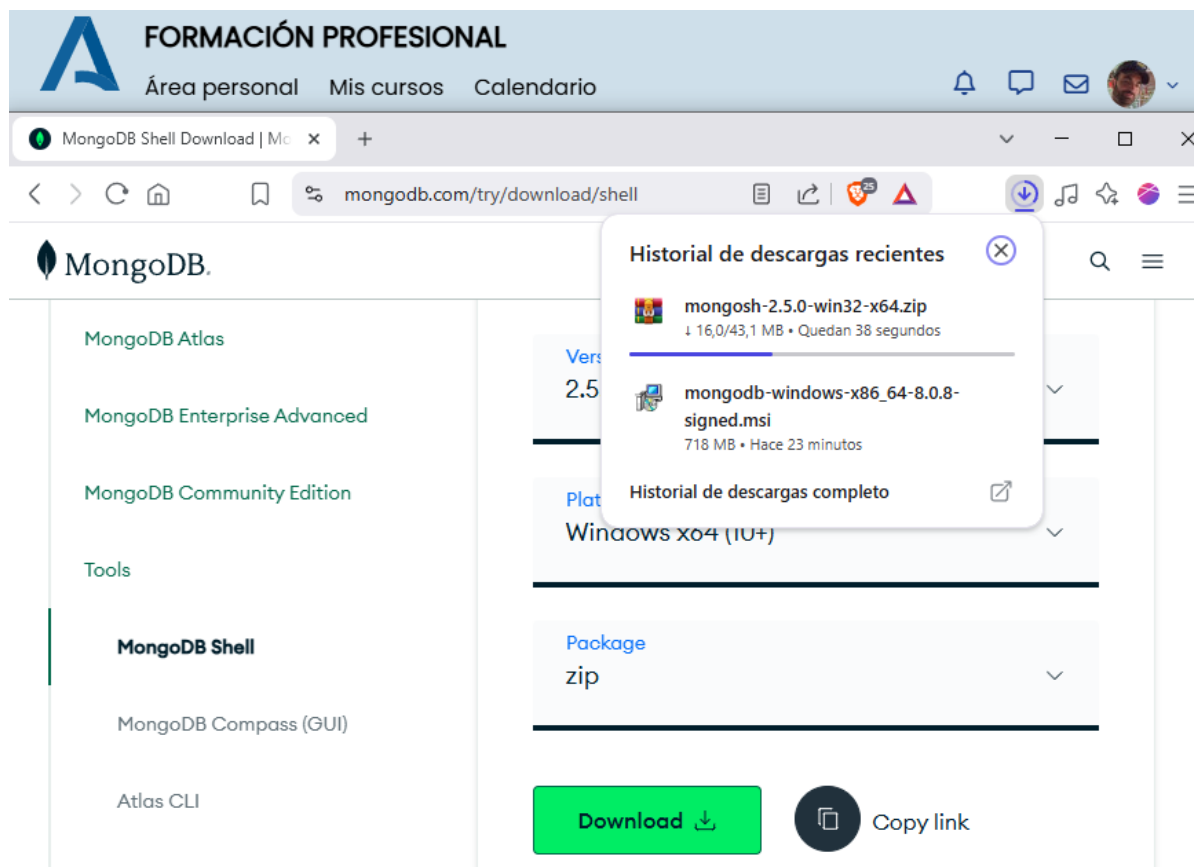
Una vez instalado se comprueba que el servicio MongoDB está corriendo en Windows 10:



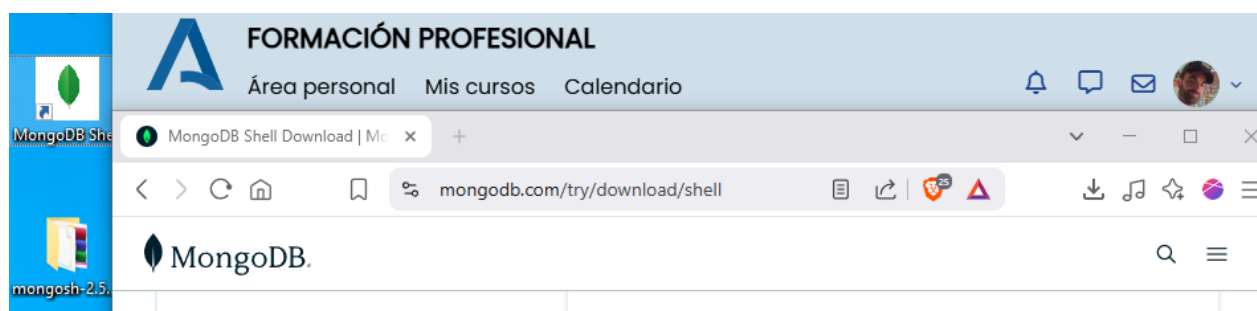
3. Ejecutamos **Compass** con la conexión que presenta por defecto. Se puede apreciar que nuestra conexión es localhost:27017 y contiene 3 bases de datos por defecto (admin, config y local):



4. Se descarga el fichero zip de la shell de la propia web oficial de MongoDB:

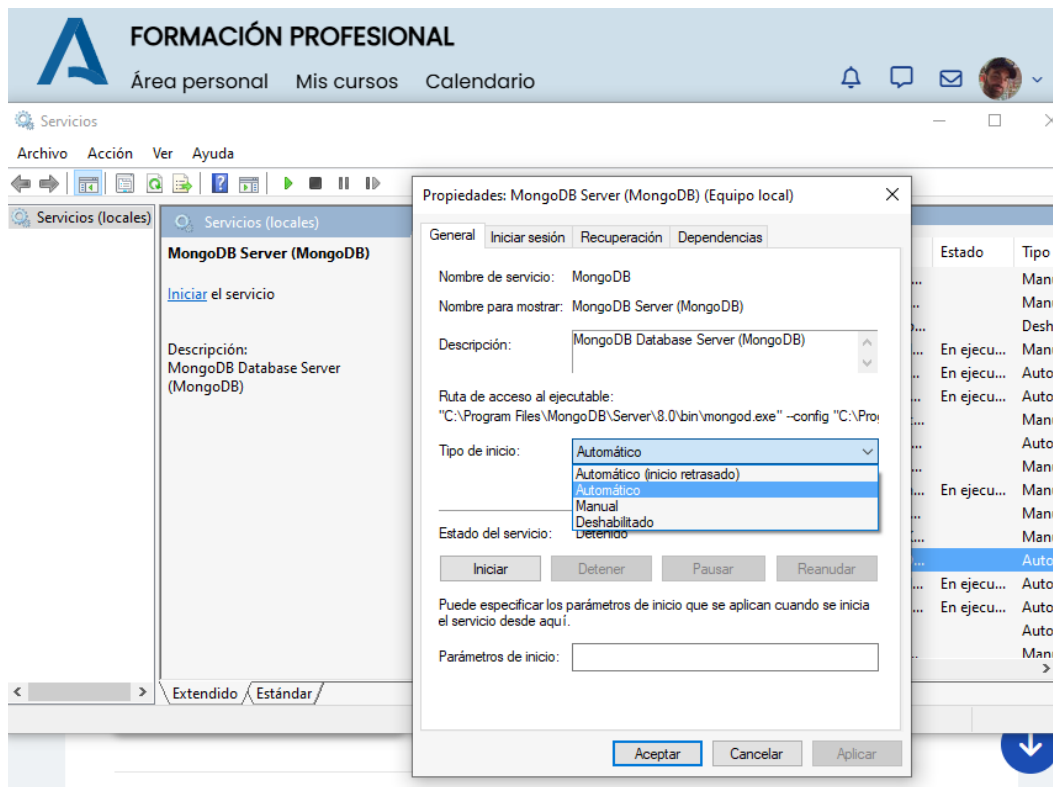


Posteriormente se descomprime en mi Escritorio y se crea acceso directo:

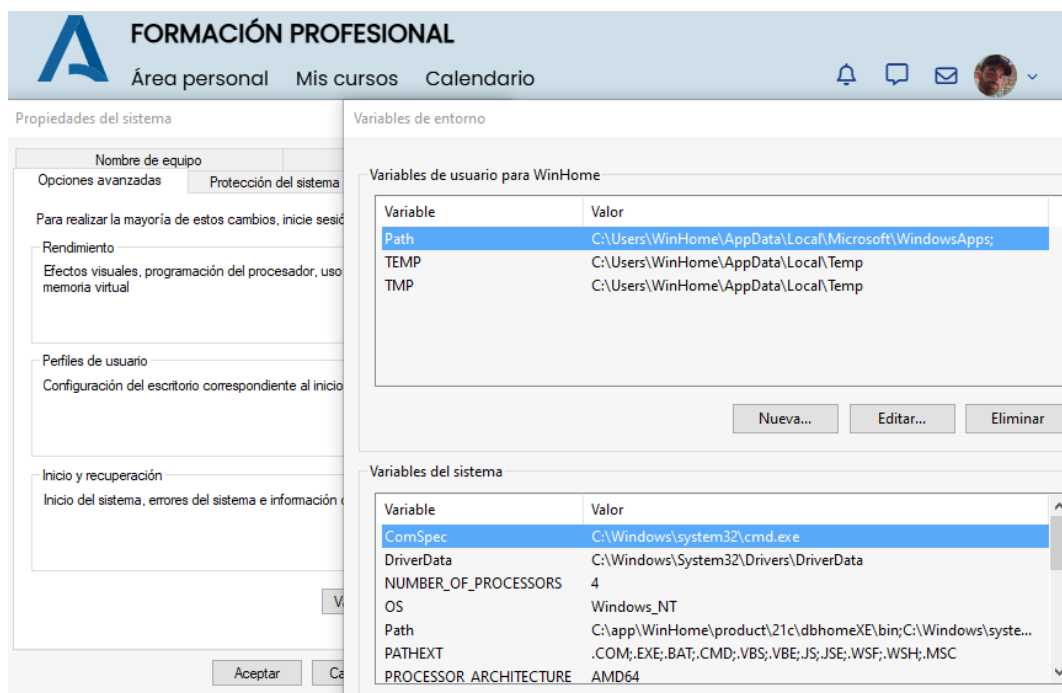


5. A continuación se configuran las variables del entorno:

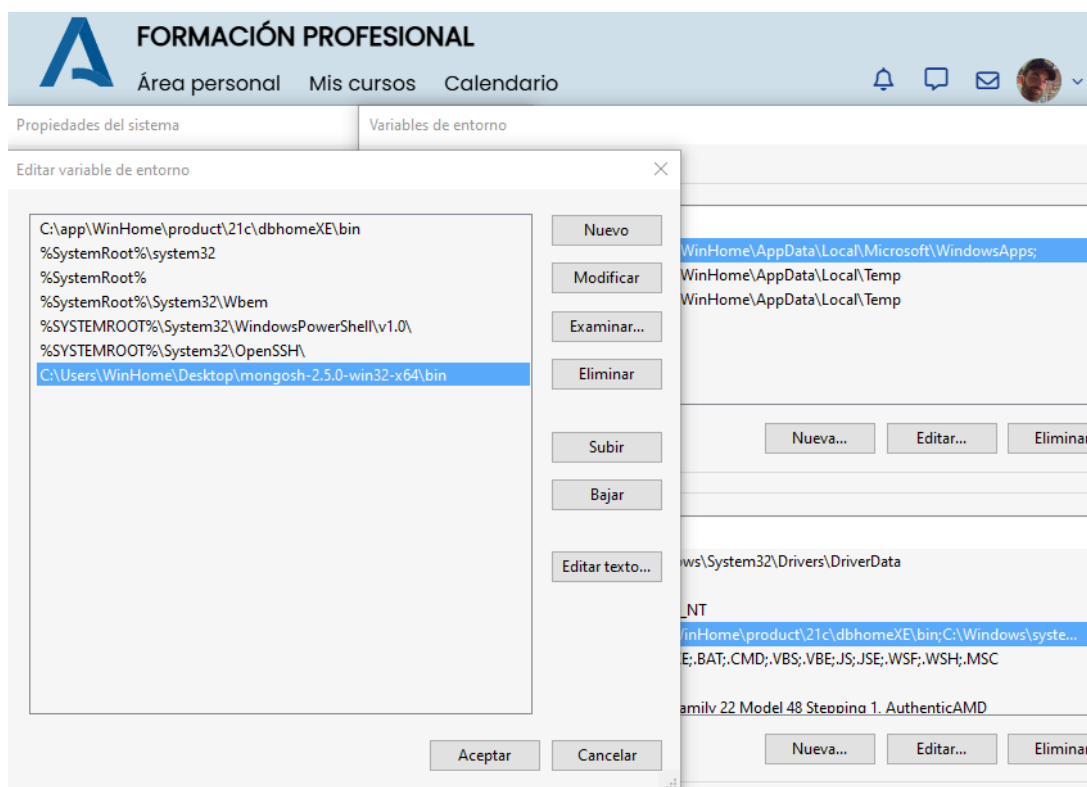
Primero de todo ponemos el servicio en modo automático para que arranque al iniciar Windows 10:



Posteriormente buscamos en Propiedades del sistema la opción Variables del entorno:



A continuación añadimos la línea de ejecución de mongosh:



Y ejecutamos la shell desde el CMD de Windows para comprobar que funciona:

```
Microsoft Windows [Versión 10.0.19044.5737]
(c) Microsoft Corporation. Todos los derechos reservados.

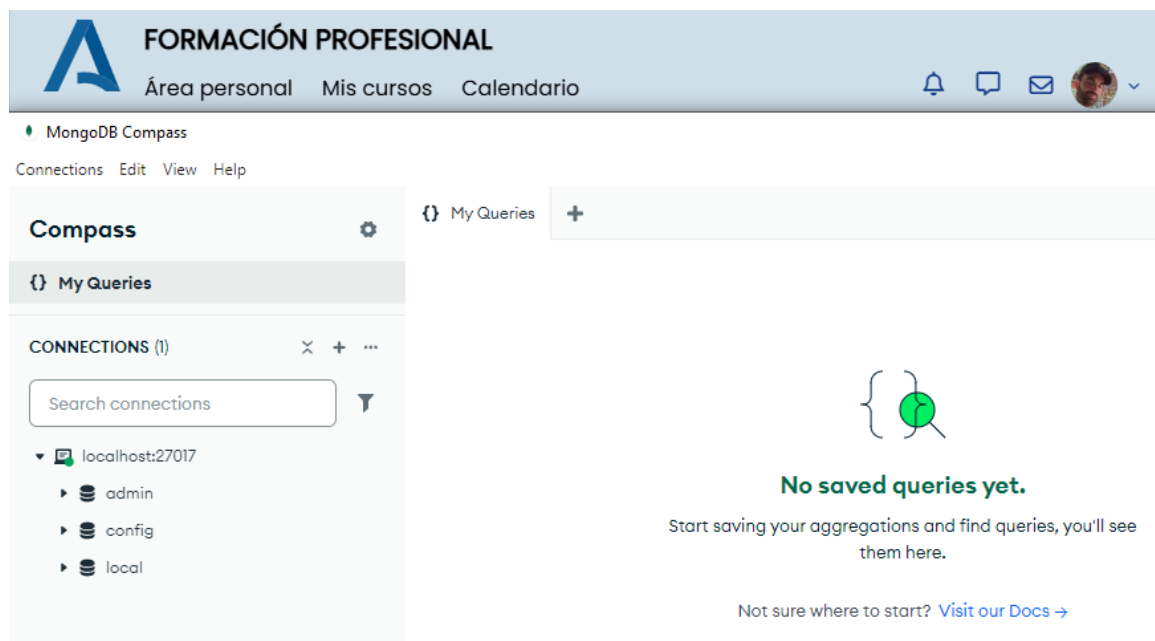
C:\Users\WinHome>mongosh
Current Mongosh Log ID: 6810f0ede1b6d44e2cb5f898
Connecting to: mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.0
Using MongoDB: 8.0.8
Using Mongosh: 2.5.0

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
2025-04-28T21:17:07.776+02:00: Access control is not enabled for the database. Read and write access to
data and configuration is unrestricted
-----

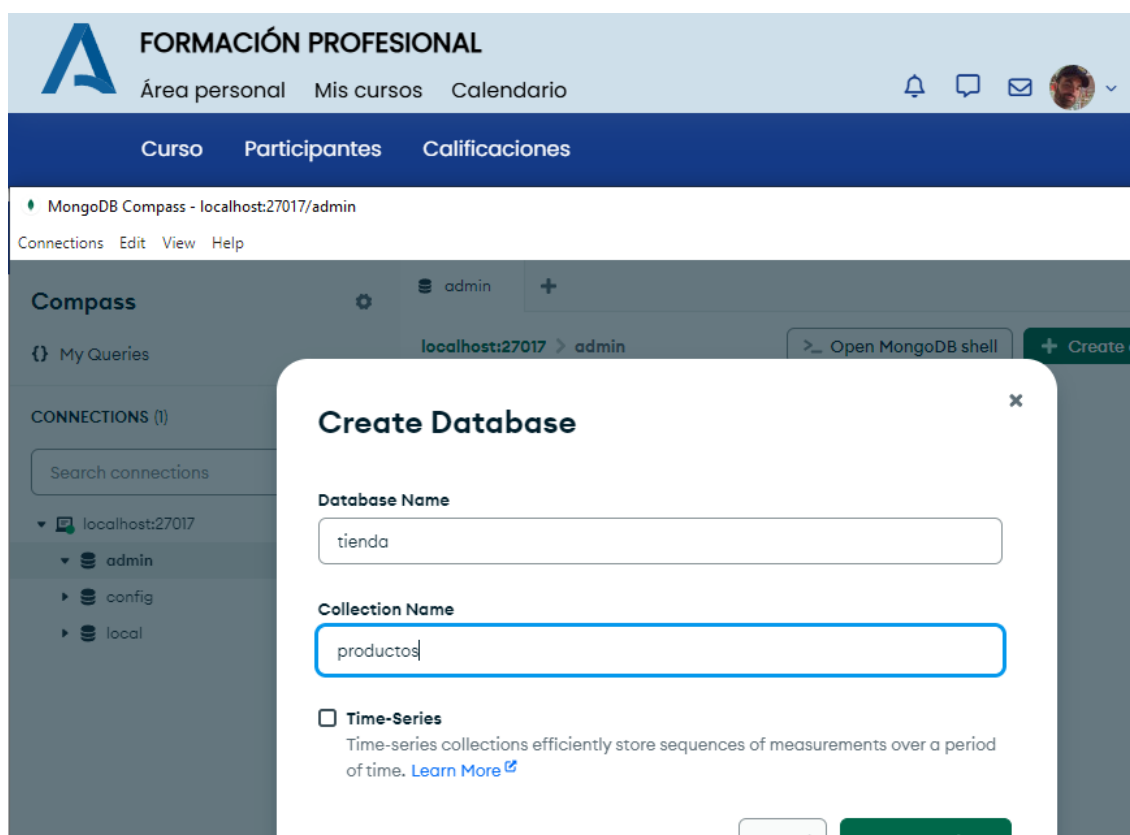
test>
```


6. Accedemos a la conexión por defecto a Compass:



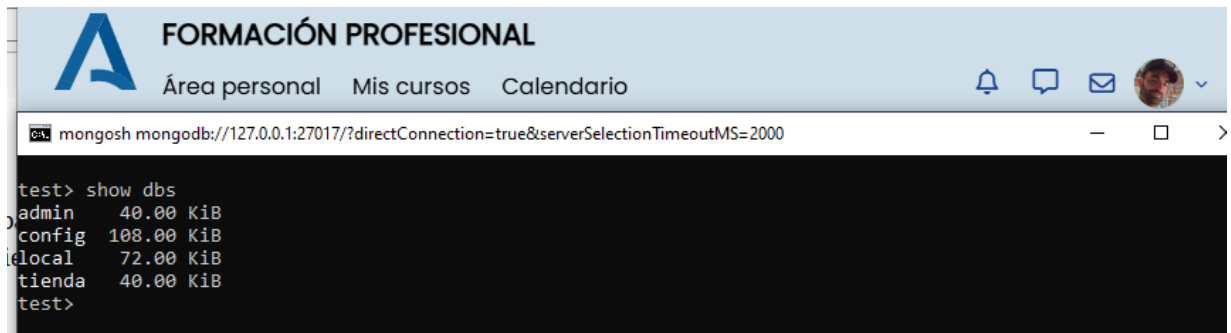
7. Creamos una base de datos llamada **tienda** desde Compass. Desde la shell la podríamos crear con el siguiente comando:

use tienda



Para verificar que se ha creado podemos introducir en la shell el siguiente comando:

show dbs

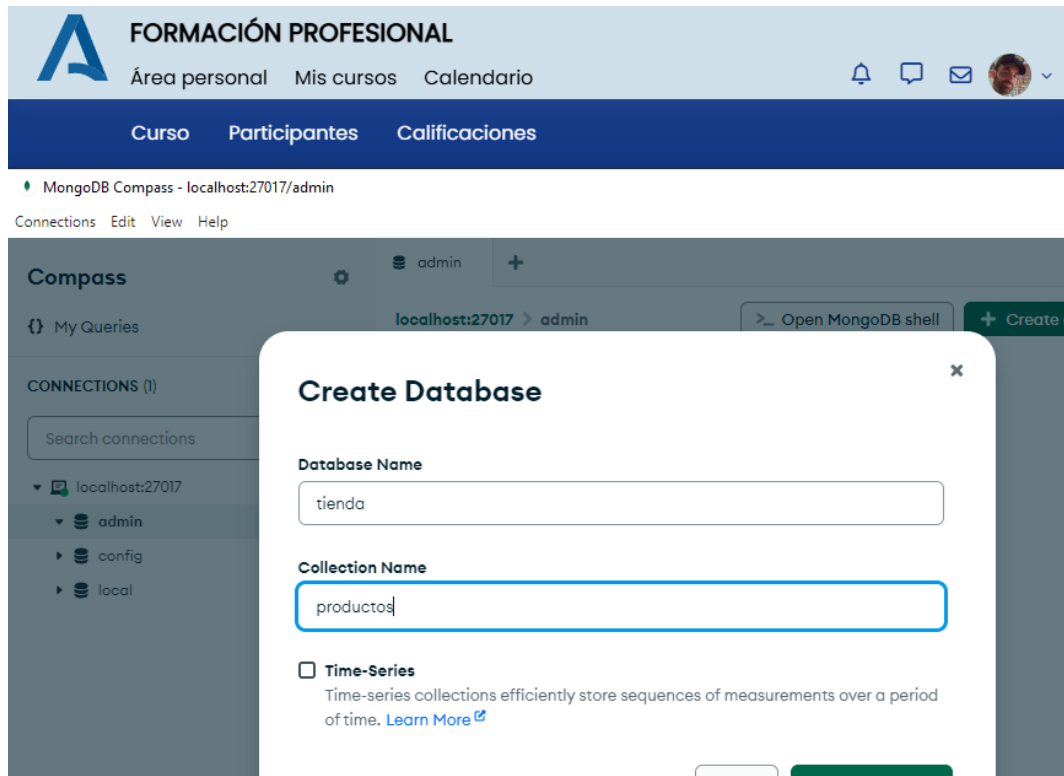


The screenshot shows a web browser window with a header for 'FORMACIÓN PROFESIONAL' and navigation links: 'Área personal', 'Mis cursos', and 'Calendario'. Below the header is a terminal window running the MongoDB shell. The terminal shows the command 'show dbs' and its output, which lists four databases and their sizes in KiB.

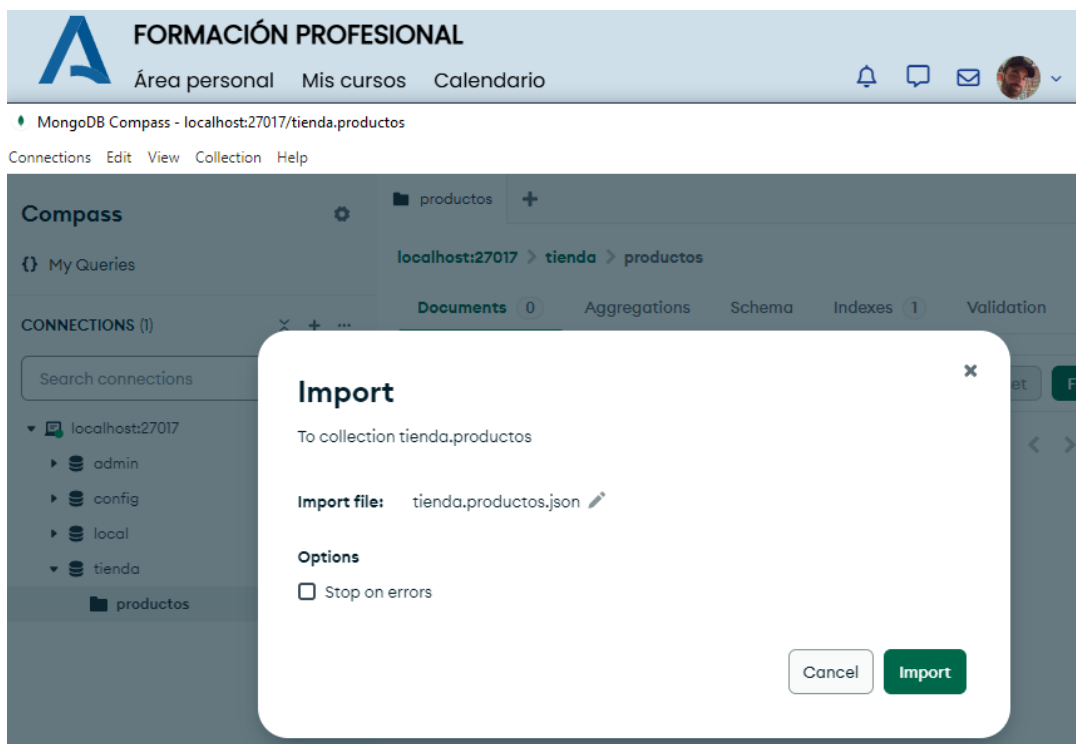
```
test> show dbs
admin      40.00 KiB
config     108.00 KiB
local      72.00 KiB
tienda     40.00 KiB
test>
```

Apartado B: Carga de datos en la base de datos y consultas.

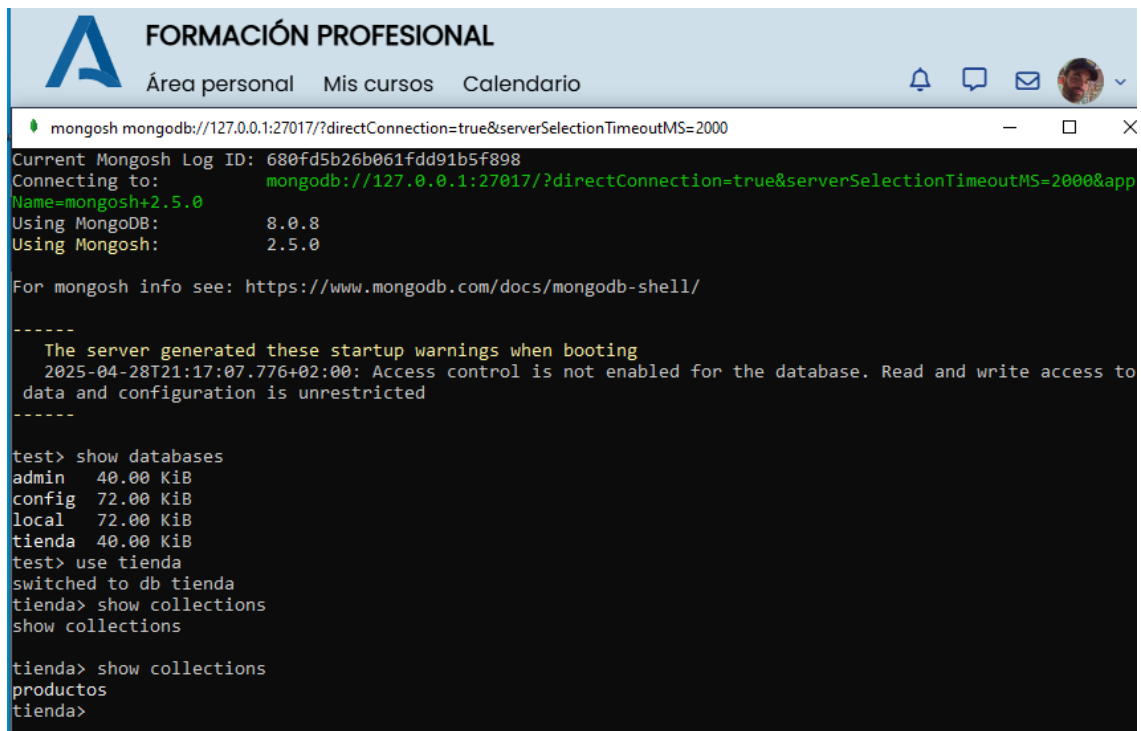
1. Creamos en la base de datos **tienda** una colección llamada **productos**. Si usamos el shell el comando sería `db.createCollection("productos")`



2. Cargamos en la colección productos los datos descargados del apartado de información de interés, con los productos de la tienda mediante Compass:



Comprobamos a través de la Shell que la base de datos **tienda** y la collection **productos** se han creado correctamente:



The screenshot shows a web browser window with the header "FORMACIÓN PROFESIONAL" and navigation links "Área personal", "Mis cursos", and "Calendario". Below the header is a terminal window titled "mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000". The terminal output shows the connection process, including the log ID, connection string, and versions of MongoDB (8.0.8) and Mongosh (2.5.0). It also displays startup warnings about access control. The user then runs the command "show databases", which lists "admin", "config", "local", and "tienda". After switching to the "tienda" database, the user runs "show collections", which lists "productos".

```
Current Mongosh Log ID: 680fd5b26b061fdd91b5f898
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&app
Name=mongosh+2.5.0
Using MongoDB:      8.0.8
Using Mongosh:      2.5.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2025-04-28T21:17:07.776+02:00: Access control is not enabled for the database. Read and write access to
  data and configuration is unrestricted
-----

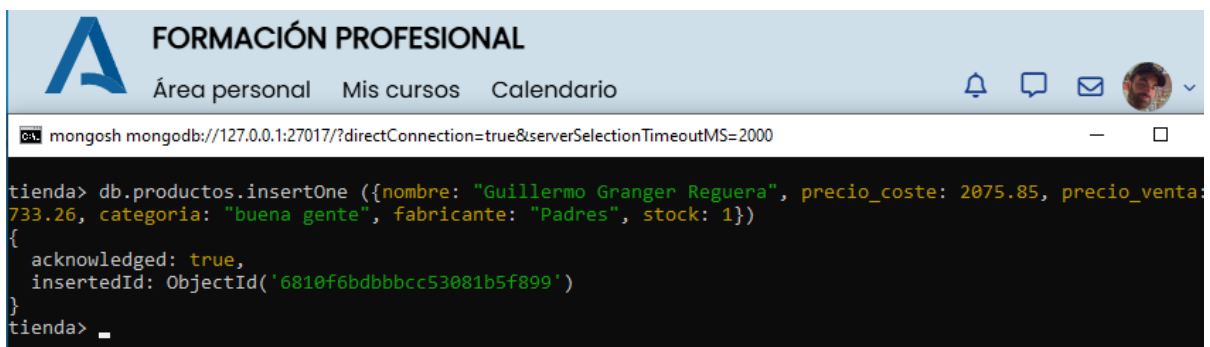
test> show databases
admin   40.00 KiB
config 72.00 KiB
local  72.00 KiB
tienda 40.00 KiB
test> use tienda
switched to db tienda
tienda> show collections
show collections

tienda> show collections
productos
tienda>
```

3. Ejecuta **la shell** de MongoDB y escribe la consulta necesaria para insertar un nuevo documento con los siguientes datos: "nombre": **tu nombre y apellidos**, precio_coste con valor 2075.85, precio_venta con valor 3733.26, categoría con valor "buena gente", fabricante con valor "Padres" y stock con valor 1.

db.productos.insertOne(

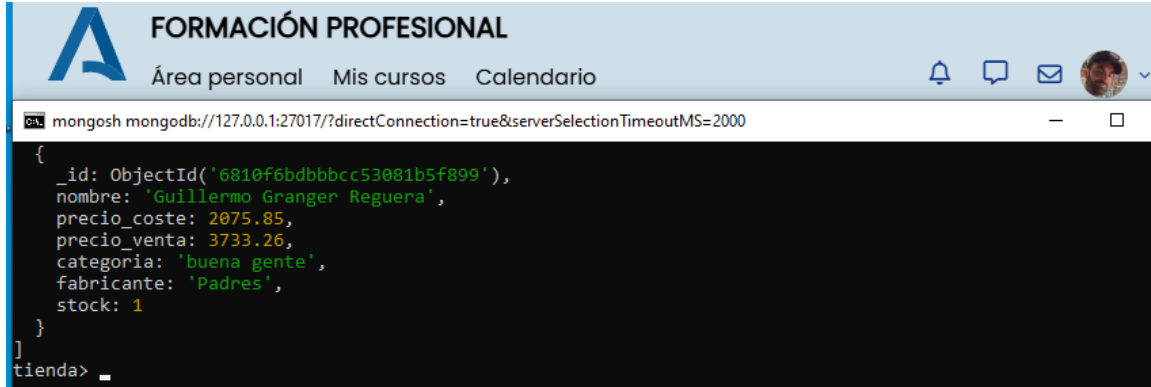
```
{
  nombre: "Guillermo Granger Reguera", precio_coste: 2075.85,
  precio_venta: 3733.26, categoría: "buena gente", fabricante: "Padres", stock: 1
}
)
```



The screenshot shows the same web browser window as before. The terminal window now shows the execution of the "insertOne" command. The output indicates that the document was successfully inserted into the "productos" collection, with an acknowledged status of true and an inserted ID of "6810f6bdbbbcc53081b5f899".

```
tienda> db.productos.insertOne ({nombre: "Guillermo Granger Reguera", precio_coste: 2075.85, precio_venta:
733.26, categoría: "buena gente", fabricante: "Padres", stock: 1})
{
  acknowledged: true,
  insertedId: ObjectId('6810f6bdbbbcc53081b5f899')
}
tienda> _
```

Comprobamos que se ha creado correctamente con `db.productos.find()`:



The screenshot shows a web browser with the header 'FORMACIÓN PROFESIONAL' and navigation links 'Área personal', 'Mis cursos', and 'Calendario'. Below the browser, a terminal window shows the command `mongo` and the output of `use tienda` and `db.productos.find()`. The output is a JSON document:

```
{
  "_id": ObjectId('6810f6bdbbbcc53081b5f899'),
  "nombre": 'Guillermo Granger Reguera',
  "precio_coste": 2075.85,
  "precio_venta": 3733.26,
  "categoria": 'buena gente',
  "fabricante": 'Padres',
  "stock": 1
}
```

4. Ejecuta en la **shell** la consulta necesaria para reemplazar el documento que acabamos de insertar y que tenga por datos: "nombre": **tus apellidos y tu nombre**, precio_coste con valor 3500, precio_venta con valor 400, categoría con valor "Good people", fabricante con valor "Padres" y stock con valor 1.

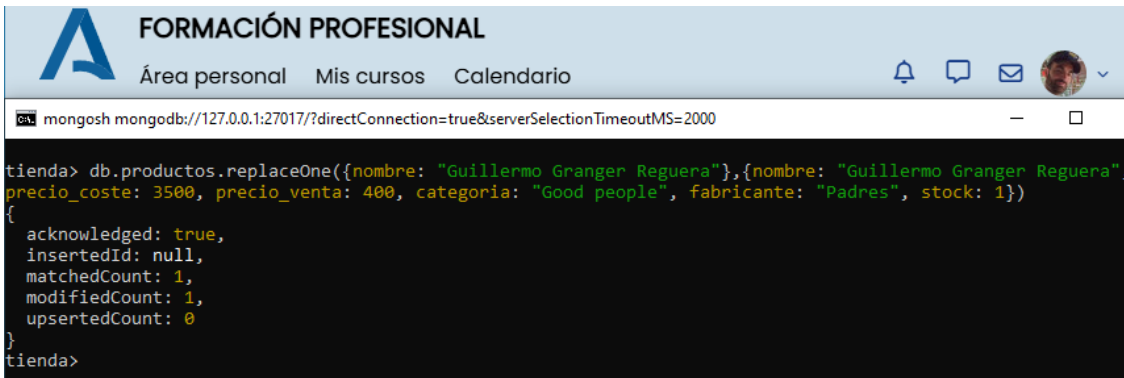
`db.productos.replaceOne(`

`{ nombre: "Guillermo Granger Reguera" },`

`{ nombre: "Guillermo Granger Reguera", precio_coste: 3500, precio_venta: 400,`

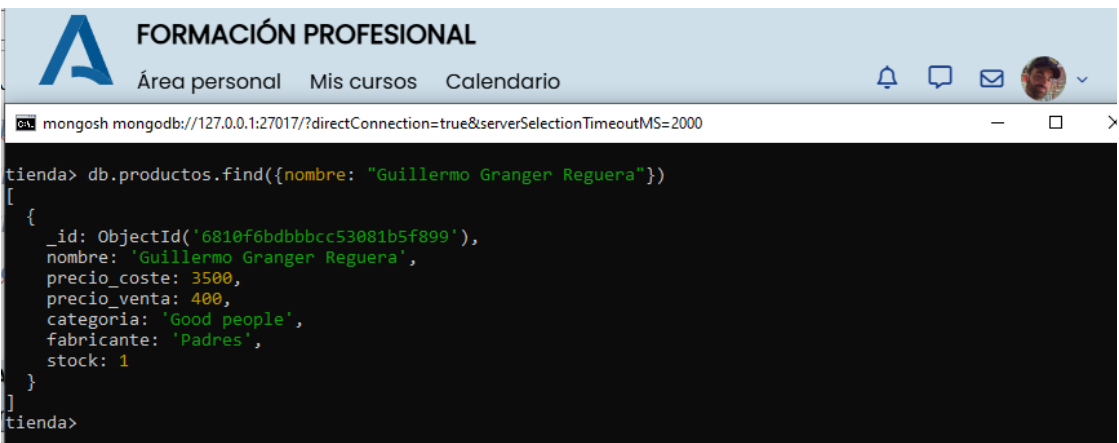
`categoría: "Good people", fabricante: "Padres", stock: 1 }`

`)`



The screenshot shows a web browser with the header 'FORMACIÓN PROFESIONAL' and navigation links 'Área personal', 'Mis cursos', and 'Calendario'. Below the browser, a terminal window shows the command `tienda> db.productos.replaceOne({nombre: "Guillermo Granger Reguera"},{nombre: "Guillermo Granger Reguera", precio_coste: 3500, precio_venta: 400, categoria: "Good people", fabricante: "Padres", stock: 1})`. The output is a JSON document:

```
{
  "acknowledged": true,
  "insertedId": null,
  "matchedCount": 1,
  "modifiedCount": 1,
  "upsertedCount": 0
}
```



The screenshot shows a web browser with the header 'FORMACIÓN PROFESIONAL' and navigation links 'Área personal', 'Mis cursos', and 'Calendario'. Below the browser, a terminal window shows the command `tienda> db.productos.find({nombre: "Guillermo Granger Reguera"})`. The output is a JSON document:

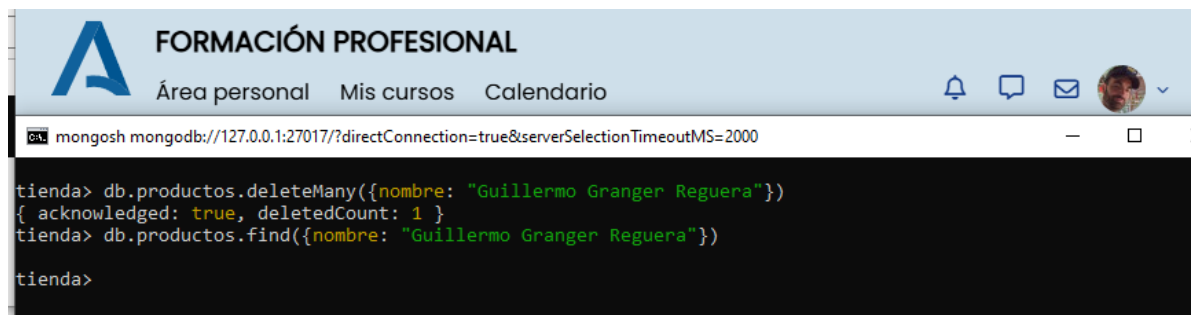
```
[
  {
    "_id": ObjectId('6810f6bdbbbcc53081b5f899'),
    "nombre": 'Guillermo Granger Reguera',
    "precio_coste": 3500,
    "precio_venta": 400,
    "categoria": 'Good people',
    "fabricante": 'Padres',
    "stock": 1
  }
]
```

5. Ejecuta en la **shell** la consulta necesaria para eliminar todos los documentos con nombre: tus apellidos y tu nombre.

db.productos.deleteMany(

{ nombre: "Guillermo Granger Reguera" }

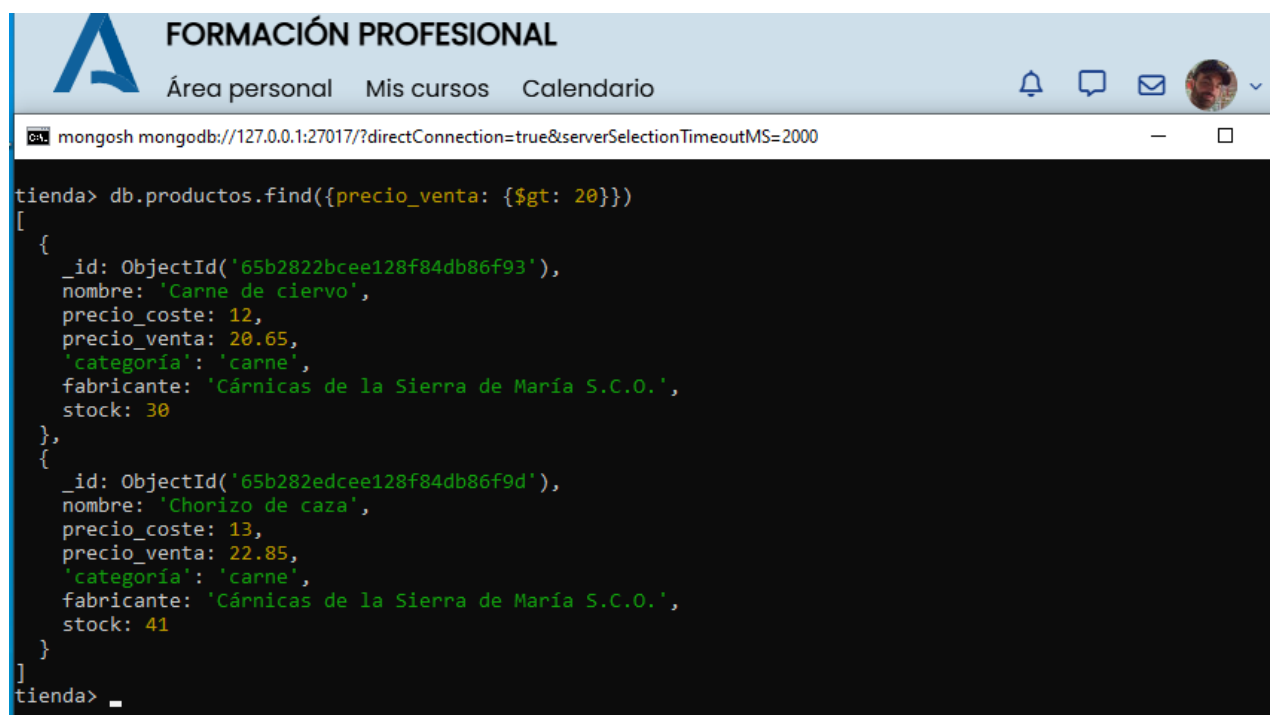
)



```
tienda> db.productos.deleteMany({nombre: "Guillermo Granger Reguera"})
{ acknowledged: true, deletedCount: 1 }
tienda> db.productos.find({nombre: "Guillermo Granger Reguera"})
tienda>
```

6. Ejecuta en la **shell** la consulta necesaria para consultar los documentos con precio de venta mayor de 20.

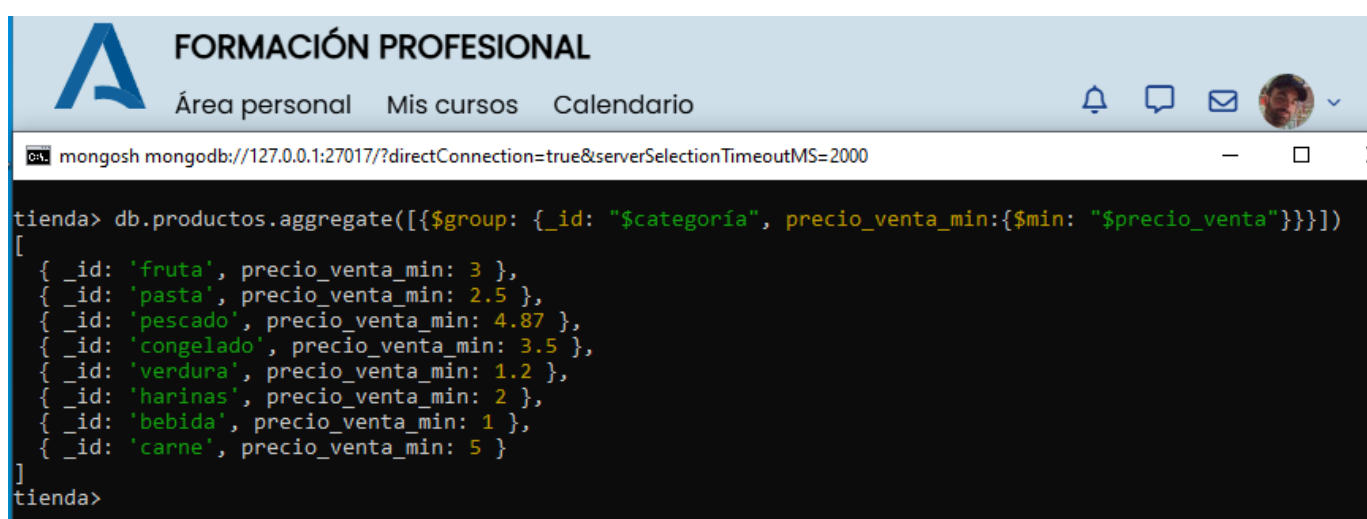
db.productos.find({precio_venta: { \$gt: 20 }})



```
tienda> db.productos.find({precio_venta: { $gt: 20 }})
[
  {
    _id: ObjectId('65b2822bcee128f84db86f93'),
    nombre: 'Carne de ciervo',
    precio_coste: 12,
    precio_venta: 20.65,
    'categoria': 'carne',
    fabricante: 'Cárnicas de la Sierra de María S.C.O.',
    stock: 30
  },
  {
    _id: ObjectId('65b282edcee128f84db86f9d'),
    nombre: 'Chorizo de caza',
    precio_coste: 13,
    precio_venta: 22.85,
    'categoria': 'carne',
    fabricante: 'Cárnicas de la Sierra de María S.C.O.',
    stock: 41
  }
]
tienda> _
```

7. Ejecuta en la **shell** la consulta necesaria para averiguar el **mínimo precio de venta** por categoría de productos.

```
db.productos.aggregate([
  { $group: {
    _id: "$categoría",
    precio_venta_minimo: { $min: "$precio_venta" } //usamos un alias
  }
}]
```



The screenshot shows a web browser window with a header for 'FORMACIÓN PROFESIONAL' and navigation links: 'Área personal', 'Mis cursos', and 'Calendario'. Below the header, a terminal window displays the MongoDB shell command and its output. The command is: `tienda> db.productos.aggregate([{$group: {_id: "$categoría", precio_venta_min:{ $min: "$precio_venta"}}}])`. The output is a JSON array of objects, each representing a product category and its minimum sale price.

Categoría	precio_venta_min
fruta	3
pasta	2.5
pescado	4.87
congelado	3.5
verdura	1.2
harinas	2
bebida	1
carne	5

Primero se ha agrupado con **\$group**, similar a “group by” de MySQL

Luego se indica por qué campo vamos a agrupar (categoría)

Finalmente se pone la condición con **\$min** para que muestre el mínimo. Usamos para almacenar dicho valor un alias llamado **precio_venta_minimo**

